

1) QUE1 =====> Best sum any tree path

```
// bestSumAnyTreePath ----->

#include <bits/stdc++.h>
using namespace std;
int dfs(int node, vector<vector<int>> &tree, vector<int> &values, int
&max_sum)
{
    int current = values[node];
    int max_upward = 0; // Maximum sum path starting from this node and
going up

    for (int child : tree[node])
    {
        int child_sum = dfs(child, tree, values, max_sum);
        max_upward = max(max_upward, child_sum);
    }

    // Calculate max path sum including this node
    int max_including_node = max(current, current + max_upward);
    max_sum = max(max_sum, max_including_node);

    // Consider paths going through this node: combine with any two
different children paths
    int sum_from_children = 0;
    if (!tree[node].empty())
    {
        vector<int> child_sums;
        for (int child : tree[node])
        {
            child_sums.push_back(dfs(child, tree, values, max_sum));
        }
        sort(child_sums.rbegin(), child_sums.rend()); // Sort descending
        sum_from_children = current + child_sums[0];
        if (child_sums.size() > 1)
        {
            sum_from_children += child_sums[1];
        }
        max_sum = max(max_sum, sum_from_children);
    }
}
```

```

    }

    return max_including_node;
}

int bestSumAnyTreePath(vector<int> parent, vector<int> values)
{
    int n = parent.size();
    vector<vector<int>> tree(n);
    int root = -1;

    // Build the tree from the parent list
    for (int i = 0; i < n; i++)
    {
        if (parent[i] == -1)
        {
            root = i;
        }
        else
        {
            tree[parent[i]].push_back(i);
        }
    }

    int max_sum = INT_MIN;
    dfs(root, tree, values, max_sum);

    return max_sum;
}

int main()
{
    vector<int> par = {-1, 0, 1, 2, 0};
    vector<int> val = {-2, 10, 10, -3, 10};

    cout << bestSumAnyTreePath(par, val) << endl;

    return 0;
}

/// 2

```

```

// long long result = 0;

// long long findMaximumPathSum(int currentNode,
//                               int previousNode,
//                               const vector<vector<int>>> &adj,
//                               const vector<int> &A)
// {
//     const vector<int> &v = adj[currentNode];
//     int maximumBranchSum1 = 0;
//     int maximumBranchSum2 = 0;
//     for (auto vtx : v)
//     {
//         if (vtx == previousNode)
//         {
//             continue;
//         }
//         long long bs = findMaximumPathSum(vtx, currentNode,
//                                           adj, A);

//         if (bs >= maximumBranchSum1)
//         {
//             maximumBranchSum2 = maximumBranchSum1;
//             maximumBranchSum1 = bs;
//         }
//         else
//         {
//             maximumBranchSum2 = max(maximumBranchSum2, bs);
//         }
//     }
//     result = max(result, A[currentNode] + maximumBranchSum1 +
// maximumBranchSum2);

//     return A[currentNode] + maximumBranchSum1;
// }

// long long bestSumAnyTreePath(vector<int> &parents, vector<int> &values)
// {
//     int n = parents.size();
//     vector<vector<int>>> mp(n);

```

```

//      for (int i = 0; i < n; i++)
//      {
//          if (parents[i] != -1)
//          {
//              mp[parents[i]].push_back(i);
//          }
//      }

//      result = 0;
//      findMaximumSumPath(0, -1, mp, values);

//      return result;
//  }

```

Ques-2 -----> Best Sum Downward Tree Path

```

// Best Sum Downward Tree Path----->
#include <bits/stdc++.h>
using namespace std;

int findmaxPath(int root, int &ans, vector<int> &parent, vector<int>
&values, vector<bool> &computed)
{
    if (computed[root])
        return values[root];
    if (parent[root] == -1)
    {
        computed[root] = true;
    }
    else
    {
        computed[root] = true;
        int parentpath = findmaxPath(parent[root], ans, parent, values,
computed);
        values[root] = max(parentpath + values[root], values[root]);
    }
}

```

```

        ans = max(ans, values[root]);
        return values[root];
    }

int bestSumDownwardTreePath(vector<int> parent, vector<int> values)
{
    int n = parent.size();
    int ans = values[0];
    vector<bool> computed(n, false);
    for (int i = 0; i < n; i++)
    {
        if (computed[i])
            continue;
        findmaxPath(i, ans, parent, values, computed);
    }
    return ans;
}

int main()
{
    vector<int> par = {-1, 0, 1, 2, 0};
    vector<int> val = {-2, 10, 10, -3, 10};

    cout << bestSumDownwardTreePath(par, val) << endl;

    return 0;
}

```

Que3 -----> Binary manipulation

```

// Binary manipulation
#include <bits/stdc++.h>

```

```

using namespace std;
int main()
{
    long long n;
    cin >> n;
    long long ans = 0;
    for (int i = 0; i < 60; i++)
    {
        long long val = pow(2, i);
        if (n & val)
        {
            ans = ((pow(2, i + 1) * 1LL - 1) - ans);
        }
    }
    cout << ans << endl;
}

```

Que4-----> Bitwise Equations

```

// bitwiseEquations
#include <bits/stdc++.h>
using namespace std;
vector<long> bitwiseEquations(vector<long> a, vector<long> b)
{
    // long long x = 0, y = 0;
    int n = a.size();
    vector<long> ans(n);
    for (int i = 0; i < n; i++)
    {
        if (a[i] < b[i])
        {

```

```

        ans[i] = 0;
        continue;
    }
    long x = 0, y = 0;
    long diff = a[i] - b[i];
    diff /= 2;
    // cout << diff << endl;
    for (int j = 0; j < 64; j++)
    {
        if (b[i] & (1 << j))
        {
            if ((diff & (1 << j)) == 0)
            {
                // x |= (1 << j);
                y |= (1 << j);
            }
            else
            {
                x = 0, y = 0;
                break;
            }
        }
        else
        {
            if ((diff & (1 << j)))
            {
                x |= (1 << j);
                y |= (1 << j);
            }
        }
    }
    ans[i] = 2 * x + 3 * y;
}

return ans;
}

int main()
{
    vector<long> a = {29, 36, 57};
    vector<long> b = {25, 18, 3};

```

```

vector<long> ans = bitwiseEquations(a, b);
for (auto x : ans)
    cout << x << " ";
return 0;
}

```

Que5----->Contiguous substrings/papercuttings

```

#include <bits/stdc++.h>
using namespace std;

long long paperCuttings(int len, vector<int> &A, vector<int> &B)
{
    set<array<int, 2>> set;
    for (int i = 0; i < A.size(); i++)
    {
        if (B[i] <= len)
        {
            set.insert({A[i], B[i]});
        }
    }
    vector<array<int, 2>> AB;
    vector<int> NA;
    for (array<int, 2> p : set)
    {
        AB.push_back(p);
        NA.push_back(p[0]);
    }
    int N = AB.size();
    long long ways = 0;
    for (int i = 0; i < N; i++)
    {
        int pairwith = N - (upper_bound(NA.begin(), NA.end(), AB[i][1]) -

```



```

        NA.begin());

    ways += pairwith;
}

return ways;
}

int main()
{
    int len = 10;
    vector<int> A = {3, 1, 2, 8, 8};
    vector<int> B = {5, 3, 7, 10, 10};
    cout << paperCuttings(len, A, B) << endl;
    return 0;
}

```

Que6 -----> Do they belong

```

#include <bits/stdc++.h>
using namespace std;

struct Point
{
    double x, y;
};

double area(Point p1, Point p2, Point p3)
{
    return std::abs((p1.x * (p2.y - p3.y) + p2.x * (p3.y - p1.y) + p3.x *
(p1.y - p2.y)) / 2.0);
}

bool isInside(Point a, Point b, Point c, Point p)
{

```

```

    double A = area(a, b, c);
    double A1 = area(b, c, p);
    double A2 = area(a, c, p);
    double A3 = area(b, a, p);
    return A == A1 + A2 + A3;
}

int pointsBelong(int x1, int y1, int x2, int y2, int x3, int y3, int xp,
int yp, int xq, int yq)
{
    Point a = {static_cast<double>(x1), static_cast<double>(y1)};
    Point b = {static_cast<double>(x2), static_cast<double>(y2)};
    Point c = {static_cast<double>(x3), static_cast<double>(y3)};
    Point p = {static_cast<double>(xp), static_cast<double>(yp)};
    Point q = {static_cast<double>(xq), static_cast<double>(yq)};

    double ab = std::sqrt(std::pow(a.x - b.x, 2) + std::pow(a.y - b.y,
2));
    double bc = std::sqrt(std::pow(c.x - b.x, 2) + std::pow(c.y - b.y,
2));
    double ac = std::sqrt(std::pow(a.x - c.x, 2) + std::pow(a.y - c.y,
2));

    int output = 0;
    if (ab + bc > ac && bc + ac > ab && ab + ac > bc)
    {
        if (isInside(a, b, c, p) && isInside(a, b, c, q))
        {
            output = 3;
        }
        else if (isInside(a, b, c, p))
        {
            output = 1;
        }
        else if (isInside(a, b, c, q))
        {
            output = 2;
        }
        else
        {
            output = 4;
        }
    }
}

```

```

    }

}

return output;
}

int main()
{
    int result = pointsBelong(2, 2, 7, 2, 5, 4, 4, 3, 7, 4);
    cout << result << endl;
    return 0;
}

```

Que7—————>Efficient harvest

```

#include <bits/stdc++.h>
using namespace std;

int maximumProfit(int m, vector<int> &earnings)
{
    int len = earnings.size();
    vector<int> reversed(len);

    for (int i = 0; i < len; i++)
    {
        reversed[i] = earnings[(i + len / 2) % len];
    }

    int currentEarnings = 0;
    for (int i = 0; i < m; i++)
    {
        currentEarnings += earnings[i] + reversed[i];
    }

    int bestProfit = currentEarnings;
}

```

```

    for (int i = 1; i < len; i++)
    {
        currentEarnings -= (earnings[i - 1] + reversed[i - 1]);
        currentEarnings += (earnings[(i + m - 1) % len] + reversed[(i + m - 1) % len]);

        bestProfit = max(bestProfit, currentEarnings);
    }

    return bestProfit;
}

int main()
{
    int m = 2;
    vector<int> earnings = {1, 5, 1, 3, 7, -3};
    cout << maximumProfit(m, earnings) << endl;
    return 0;
}

```

Ques8-----> Element swapping

```

#include <bits/stdc++.h>
using namespace std;
long long getMaximumStrength(vector<int> a)
{
    int n = a.size();
    vector<vector<int>> dp(n, vector<int>(2));
    dp[0][0] = a[0];
    dp[0][1] = a[0];

    for (long long i = 1; i < n; i++)
    {
        dp[i][0] = max(dp[i - 1][0], dp[i - 1][1]) + a[i] * (i + 1);
    }
}

```

```

        dp[i][1] = a[i] * i + a[i - 1] * (i + 1) + (i - 2 >= 0 ? max(dp[i - 2][0], dp[i - 2][1]) : 0);
    }

    cout << max(dp[n - 1][0], dp[n - 1][1]) << endl;
    return max(dp[n - 1][0], dp[n - 1][1]);
}

int main()
{
    vector<int> arr = {1, 9, 7, 3, 2};
    getMaximumStrength(arr);
    return 0;
}

```

Que9----->Intelligent Substring

```

#include <bits/stdc++.h>
using namespace std;

int getSpecialSubstring(const std::string &s, int k, const std::string &cv)
{
    int n = s.length(), ml = 0, l = 0, nc = 0;

    for (int r = 0; r < n; r++)
    {
        if (cv[s[r] - 'a'] == '0')
        {
            nc++;
        }

        while (nc > k)
        {
            if (cv[s[l] - 'a'] == '0')

```

```

        {
            nc--;
        }
        l++;
    }

    ml = std::max(ml, r - l + 1);
}

return ml;
}

int main()
{
    std::string s = "giraffe";
    int k = 2;
    std::string cv = "0111100111111111101111111";
    int result = getSpecialSubstring(s, k, cv);
    std::cout << "Length of the longest substring containing at most " <<
k << " special characters: " << result << std::endl;
    return 0;
}

```

Que.10 -----> kmeans

```

#include <bits/stdc++.h>
using namespace std;

int getMaximumDistance(vector<int> location, int k)
{
    int n = location.size();
    sort(location.begin(), location.end());

    int maxi = location[n - 1] - location[0];

    int left = 0, right = maxi;

```

```

while (left < right)

{
    int mid = left + (right - left) / 2;
    int center = 1;
    int prev = location[0];
    for (int i = 1; i < n; i++)

    {
        if (location[i] - prev > mid * 2)

        {
            center++;
            prev = location[i];
        }
    }

    if (center <= k)

    {
        right = mid;
    }
    else
    {
        left = mid + 1;
    }
}

return left;
}

int main()
{
    vector<int> location = {4, 1, 6, 7, 2};
    int k = 2;
    cout << "Maximum distance: " << getMaximumDistance(location, k) <<
endl;
    return 0;
}

```

Que11----->Quiz Competition

```
#include <bits/stdc++.h>
using namespace std;
vector<int> findMinTeamLengths(int tCount, vector<int> &tList)
{
    int n = tList.size();
    vector<int> res(n, -1);

    unordered_map<int, int> tMap;
    int dCount = 0;
    int l = 0;

    for (int r = 0; r < n; r++)
    {
        int curT = tList[r];

        if (tMap[curT] == 0)
        {
            dCount++;
        }
        tMap[curT]++;

        while (dCount == tCount)
        {
            res[l] = r - l + 1;
            int lT = tList[l];
            tMap[lT]--;
            if (tMap[lT] == 0)
            {
                dCount--;
            }
            l++;
        }
    }
}
```



```

        return res;
    }

int main()
{
    int tCount = 3;
    vector<int> tList = {1, 2, 3, 2, 1};
    vector<int> minLengths = findMinTeamLengths(tCount, tList);
    for (int len : minLengths)
        cout << len << " ";
    return 0;
}

```

Que12-----> Rearrange students

```

#include <bits/stdc++.h>
using namespace std;

long long minCost(vector<int> &a, vector<int> &b)
{
    map<int, int> mp;
    int n = a.size();
    int mini = INT_MAX;
    for (int i = 0; i < n; i++)
    {
        mp[a[i]]++;
        mp[b[i]]--;
        mini = min(mini, a[i]);
        mini = min(mini, b[i]);
    }
    vector<int> x;
    for (auto it : mp)
    {
        int t = it.second;
        if (t % 2 == 1)
            return -1;
        else

```

```

        {
            for (int i = 0; i < abs(t) / 2; i++)
            {
                x.push_back(it.first);
            }
        }
    }
    long long ans = 0;
    int m = x.size();
    for (int i = 0; i < m / 2; i++)
    {
        ans += min(x[i], 2 * mini);
    }
    return ans;
}

int main()
{
    vector<int> a = {4, 2, 2, 2};
    vector<int> b = {1, 4, 1, 2};
    cout << minCost(a, b) << endl; // Output: 6
    return 0;
}

```

Que13----->Social Network

```

#include <bits/stdc++.h>
using namespace std;
void socialGraph(vector<int> counts)
{
    int n = counts.size();
    map<int, vector<int>> mp;
    vector<int> vis(n + 1, 0);
    for (int i = n - 1; i >= 0; i--)
        mp[counts[i]].push_back(i);
}

```

```

        for (int i = 0; i < n; i++)
        {
            if (!vis[i])
            {
                for (int j = 0; j < counts[i]; j++)
                {
                    cout << mp[counts[i]].back() << " ";
                    vis[mp[counts[i]].back()] = 1;
                    mp[counts[i]].pop_back();
                }
                cout << endl;
            }
        }
    }

int main()
{
    vector<int> counts = {2, 2, 2, 2};
    socialGraph(counts);
    return 0;
}

```

Que14—————> string subsequences

```

// String Subsequences
// Given two strings, determine the number of times the first one appears
as a subsequence in the second one. A subsequence is created by eliminating
any number of characters
// from a string (possibly 0) without changing the order of the characters
retained.

#include <bits/stdc++.h>
using namespace std;

int count_subsequences(const std::string &s1, const std::string &s2)

```

```

{
    int m = s1.length(), n = s2.length();
    std::vector<std::vector<int>>> dp(m + 1, std::vector<int>(n + 1, 0));

    for (int j = 0; j <= n; ++j)
    {
        dp[0][j] = 1;
    }

    for (int i = 1; i <= m; ++i)
    {
        for (int j = 1; j <= n; ++j)
        {
            if (s1[i - 1] == s2[j - 1])
            {
                dp[i][j] = dp[i][j - 1] + dp[i - 1][j - 1];
            }
            else
            {
                dp[i][j] = dp[i][j - 1];
            }
        }
    }

    return dp[m][n];
}

int main()
{
    std::string s1 = "ABC";
    std::string s2 = "ABCBABC";
    int count = count_subsequences(s1, s2);
    std::cout << "Number of distinct subsequences: " << count <<
std::endl;
    return 0;
}

```

Que15-----> ttl cache

```
#include <bits/stdc++.h>
using namespace std;
#include <ext/pb_ds/assoc_container.hpp>
#include <ext/pb_ds/tree_policy.hpp>
using namespace __gnu_pbds;

typedef tree<int, null_type, less_equal<int>, rb_tree_tag,
tree_order_statistics_node_update> pbds;
vector<int> ttl_cache(int n, vector<vector<int>> entries, vector<int>
queries)
{
    int q = queries.size();

    vector<pair<int, int>> arr;
    for (int i = 0; i < q; i++)
    {
        arr.push_back({queries[i], i});
    }
    sort(arr.begin(), arr.end());

    vector<int> res(q);
    pbds st;

    int i = 0;
    int x = arr[i].first;
    sort(entries.begin(), entries.end());
    for (auto &v : entries)
    {
        while (v[0] > x)
        {
```

```

        res[arr[i].second] = st.size() - st.order_of_key(x);
        i++;
        if (i == q)
            return res;
        x = arr[i].first;
    }
    st.insert(v[0] + v[1]);
}

for (; i < q; i++)
{
    x = arr[i].first;
    res[arr[i].second] = st.size() - st.order_of_key(x);
}
return res;
}

// get Cache size
// vector<int> getCacheSize(vector<vector<int>> d, vector<int> q)
// {
//     map<int, int> time_map;
//     time_map[0] = 0;
//     time_map[1000000010] = 0;
//     for (auto i : d)
//     {
//         time_map[i[0]]++;
//         time_map[i[0] + i[1] + 1]--;
//     }
//     int pre = 0;
//     for (auto &[f, s] : time_map)
//     {
//         s += pre;
//         pre = s;
//     }
//     vector<int> ans;
//     for (auto it : q)
//     {
//         auto it = time_map.upper_bound(i);
//         it--;
//         ans.push_back(it->second);
//     }
// }

```

```

//      }
//      return ans;
//  }

int main()
{
    int n = 3;
    vector<vector<int>> entries = {{105231, 183}, {105334, 34}, {105198,
543}};
    vector<int> queries = {105338, 105410};
    vector<int> ans = ttl_cache(n, entries, queries);
    for (auto &x : ans)
        cout << x << " ";
    return 0;
}

```

que16—————> valid bst permutations

```

#include <bits/stdc++.h>
#define ll long long
using namespace std;
const ll mod = 1e8 + 7;

ll multmod(ll a, ll b)
{
    ll result = 0;
    while (b)
    {
        if (b & 1)
            result = (result + a) % mod;
        a = (a + a) % mod;
        b >>= 1;
    }
    return result;
}

```

```

}

ll addmod(ll a, ll b)
{
    a = a % mod;
    b = b % mod;
    return ((a + b) % mod) + mod) % mod;
}

vector<int> numBST(vector<int> nodeValues)
{
    ll maxVal = *max_element(nodeValues.begin(), nodeValues.end());
    vector<unsigned ll> dp(maxVal + 1);
    dp[0] = 1;
    dp[1] = 1;
    for (ll i = 2; i <= maxVal; i++)
    {
        unsigned ll res = 0;
        for (ll j = 0; j < i; j++)
            res = addmod(res, multmod(dp[j], dp[i - j - 1]));
        dp[i] = res % mod;
    }

    vector<int> ans(nodeValues.size());
    for (ll i = 0; i < nodeValues.size(); i++)
    {
        ans[i] = dp[nodeValues[i]] % mod;
    }
    return ans;
}

int main()
{
    ll n = 5;
    vector<int> nodeValues = {1, 2, 3, 4, 100};

    vector<int>
        ans = numBST(nodeValues);
    for (auto it : ans)
        cout << it << " ";
}

```



```
    return 0;  
}
```

Que17 -----> Get the groups

```
#include <iostream>  
#include <vector>  
#include <string>  
#include <unordered_map>  
#include <bits/stdc++.h>  
  
using namespace std;  
  
class DisjointSet  
{  
public:  
    vector<int> par, size;  
  
    DisjointSet(int n)  
    {  
        par.resize(n + 1);  
        size.resize(n + 1);  
  
        for (int i = 0; i <= n; i++)  
        {  
            par[i] = i;  
            size[i] = 1;  
        }  
    }  
  
    int find(int x)
```

```

{
    if (par[x] == x)
        return x;

    return par[x] = find(par[x]);
}

void unionSet(int x, int y)
{
    int p1 = find(x), p2 = find(y);

    if (p1 == p2)
        return;

    if (size[p1] < size[p2])
    {
        par[p1] = p2;
        size[p2] += size[p1];
    }
    else
    {
        par[p2] = p1;
        size[p1] += size[p2];
    }
}

int getSize(int x)
{
    return size[find(x)];
}
};

vector<int> getTheGroups(int n, vector<string> queryType, vector<int>
students1, vector<int> students2)
{
    DisjointSet uf(n + 1); // UnionFind for 1-based index
    vector<int> result;
    int m = queryType.size();
    for (int i = 0; i < m; ++i)
    {
        if (queryType[i] == "Friend")
        {
            uf.unionSet(students1[i], students2[i]);

```

```

    }
    else if (queryType[i] == "Total")
    {
        int root1 = uf.find(students1[i]);
        int root2 = uf.find(students2[i]);
        if (root1 == root2)
        {
            result.push_back(uf.getSize(students1[i]));
        }
        else
        {
            result.push_back(uf.getSize(students1[i]) +
uf.getSize(students2[i]));
        }
    }
}

return result;
}

int main()
{
    int n = 3;
    vector<string> queryType = {"Friend", "Total"};
    vector<int> students1 = {1, 2};
    vector<int> students2 = {2, 3};

    vector<int> result = getTheGroups(n, queryType, students1, students2);
    for (int x : result)
    {
        cout << x << endl;
    }

    return 0;
}

```